

Airport Management Information System – Ercan Airport

Course: CMPE343 Database Management Systems and Programming I
Term Project

Prepared by: Matarr Barrow 22421230, Alpha Samura 22019986



CYPRUS INTERNATIONAL UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING

Table of Contents

1. Introduction
2. Entity Relationship Diagram (ERD)
3. Relational Data Model
4. Data Definition Language (DDL)
5. Data Manipulation Language (DML) – Sample Data
6. Screenshots of All Tables
7. Queries for Management Statistics (15 Queries)
8. Additional Features
9. GitHub Repository
10. Conclusion

1. Introduction

This project implements a **relational database management system** for Ercan Airport. The system stores and manages information about airplanes, hangars, tests, technicians, pilots, controllers, maintenance records, flight schedules, and suppliers. The design meets all project specifications and extends the minimal requirements with real-life entities.

Key features:

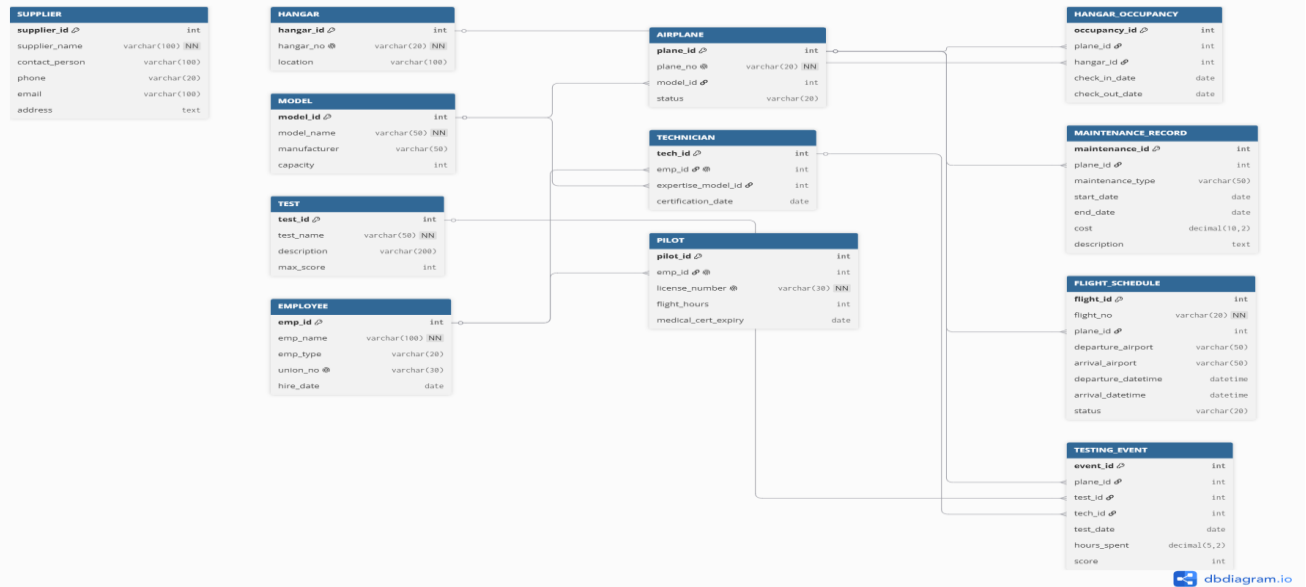
- Track airplanes, their models, and current status.
- Record hangar occupancy with check-in/check-out dates.
- Manage test definitions and testing events (which technician tested which airplane, when, hours spent, score).
- Store employee data (technicians, controllers, pilots) with union membership.
- Additional tables: PILOT, MAINTENANCE_RECORD, FLIGHT_SCHEDULE, SUPPLIER.

Assumptions:

- Each airplane belongs to exactly one model.
- A technician can be expert in one or more models (one-to-many relationship via `expertise_model_id`).
- Each employee has a unique union number.
- Testing event score is between 0 and `max_score` (enforced by application logic).

2. Entity Relationship Diagram (ERD)

The ERD was designed using dbdiagram.io and contains **12 tables**, exceeding the minimum requirement of 8.



Entity	Relationship	Entity
MODEL	one-to-many	AIRPLANE
AIRPLANE	one-to-many	HANGAR_OCCUPANCY
HANGAR	one-to-many	HANGAR_OCCUPANCY
AIRPLANE	one-to-many	TESTING_EVENT
TEST	one-to-many	TESTING_EVENT
TECHNICIAN	one-to-many	TESTING_EVENT
EMPLOYEE	one-to-one	TECHNICIAN
EMPLOYEE	one-to-one	PILOT
MODEL	one-to-many	TECHNICIAN (expertise)
AIRPLANE	one-to-many	MAINTENANCE_RECORD
AIRPLANE	one-to-many	FLIGHT_SCHEDULE

3. Relational Data Model

The relational schema (primary keys underlined, foreign keys arrowed):

MODEL(**model_id**, model_name, manufacturer, capacity)

AIRPLANE(**plane_id**, plane_no, model_id → MODEL, status)

HANGAR(**hangar_id**, hangar_no, location)

HANGAR_OCCUPANCY(**occupancy_id**, plane_id → AIRPLANE, hangar_id → HANGAR, check_in_date, check_out_date)

TEST(**test_id**, test_name, description, max_score)

EMPLOYEE(**emp_id**, emp_name, emp_type, union_no, hire_date)

TECHNICIAN(**tech_id**, emp_id → EMPLOYEE, expertise_model_id → MODEL, certification_date)

PILOT(**pilot_id**, emp_id → EMPLOYEE, license_number, flight_hours, medical_cert_expiry)

TESTING_EVENT(**event_id**, plane_id → AIRPLANE, test_id → TEST, tech_id → TECHNICIAN, test_date, hours_spent, score)

MAINTENANCE_RECORD(**maintenance_id**, plane_id → AIRPLANE, maintenance_type, start_date, end_date, cost, description)

FLIGHT_SCHEDULE(**flight_id**, flight_no, plane_id → AIRPLANE, departure_airport, arrival_airport, departure_datetime, arrival_datetime, status)

SUPPLIER(**supplier_id**, supplier_name, contact_person, phone, email, address)

4. Data Definition Language (DDL)

The complete DDL script is provided in `ercan_schema.sql` (or `ERCAN AIRPORT DATABASE (1).sql`). Below is the summary of constraints:

- **Primary Keys** on all *_id columns.
- **Foreign Keys** with ON DELETE default behaviour (no cascade – maintain integrity).
- **UNIQUE** constraints: plane_no, hangar_no, union_no, license_number.
- **CHECK** constraint on emp_type (values: 'TECHNICIAN', 'CONTROLLER', 'PILOT').

- **DEFAULT** values: status in AIRPLANE = 'ACTIVE', flight_hours = 0, status in FLIGHT_SCHEDULE = 'SCHEDULED'.

Screenshot of DDL execution in MySQL:

<pre> 1 CREATE DATABASE `ircan_airport_management`; 2 USE `ircan_airport_management`; 3 4 CREATE TABLE MODEL (5 model_id INT PRIMARY KEY AUTO_INCREMENT, 6 model_name VARCHAR(50) NOT NULL, 7 manufacturer VARCHAR(50), 8 capacity INT 9); 10 11 CREATE TABLE AIRPLANE (12 plane_id INT PRIMARY KEY AUTO_INCREMENT, 13 plane_no VARCHAR(20) UNIQUE NOT NULL, 14 model_id INT, 15 status VARCHAR(20) DEFAULT 'ACTIVE', 16 FOREIGN KEY (model_id) REFERENCES MODEL(model_id) 17); 18 19 CREATE TABLE HANGAR (20 hangar_id INT PRIMARY KEY AUTO_INCREMENT, 21 hangar_no VARCHAR(20) UNIQUE NOT NULL, 22 location VARCHAR(100) 23); 24 25 CREATE TABLE HANGAR_OCCUPANCY (26 occupancy_id INT PRIMARY KEY AUTO_INCREMENT, 27 plane_id INT, 28 hangar_id INT, 29 check_in_date DATE, 30 check_out_date DATE, 31 FOREIGN KEY (plane_id) REFERENCES AIRPLANE(plane_id), 32 FOREIGN KEY (hangar_id) REFERENCES HANGAR(hangar_id) 33); </pre>	<pre> 1 CREATE DATABASE `ircan_airport_management`; 2 USE `ircan_airport_management`; 3 4 CREATE TABLE MODEL (5 model_id INT PRIMARY KEY AUTO_INCREMENT, 6 model_name VARCHAR(50) NOT NULL, 7 manufacturer VARCHAR(50), 8 capacity INT 9); 10 11 CREATE TABLE AIRPLANE (12 plane_id INT PRIMARY KEY AUTO_INCREMENT, 13 plane_no VARCHAR(20) UNIQUE NOT NULL, 14 model_id INT, 15 status VARCHAR(20) DEFAULT 'ACTIVE', 16 FOREIGN KEY (model_id) REFERENCES MODEL(model_id) 17); 18 19 CREATE TABLE HANGAR (20 hangar_id INT PRIMARY KEY AUTO_INCREMENT, 21 hangar_no VARCHAR(20) UNIQUE NOT NULL, 22 location VARCHAR(100) 23); 24 25 CREATE TABLE HANGAR_OCCUPANCY (26 occupancy_id INT PRIMARY KEY AUTO_INCREMENT, 27 plane_id INT, 28 hangar_id INT, 29 check_in_date DATE, 30 check_out_date DATE, 31 FOREIGN KEY (plane_id) REFERENCES AIRPLANE(plane_id), 32 FOREIGN KEY (hangar_id) REFERENCES HANGAR(hangar_id) 33); </pre>
<pre> 68 CREATE TABLE TESTING_EVENT (69 event_id INT PRIMARY KEY AUTO_INCREMENT, 70 plane_id INT, 71 test_id INT, 72 tech_id INT, 73 test_date DATE, 74 hours_spent DECIMAL(5,2), 75 score INT, 76 FOREIGN KEY (plane_id) REFERENCES AIRPLANE(plane_id), 77 FOREIGN KEY (test_id) REFERENCES TEST(test_id), 78 FOREIGN KEY (tech_id) REFERENCES TECHNICIAN(tech_id) 79); 80 81 CREATE TABLE MAINTENANCE_RECORD (82 maintenance_id INT PRIMARY KEY AUTO_INCREMENT, 83 plane_id INT, 84 maintenance_type VARCHAR(50), 85 start_date DATE, 86 end_date DATE, 87 cost DECIMAL(10,2), 88 description TEXT, 89 FOREIGN KEY (plane_id) REFERENCES AIRPLANE(plane_id) 90); 91 </pre>	<pre> 92 CREATE TABLE FLIGHT_SCHEDULE (93 flight_id INT PRIMARY KEY AUTO_INCREMENT, 94 flight_no VARCHAR(20) NOT NULL, 95 plane_id INT, 96 departure_airport VARCHAR(50), 97 arrival_airport VARCHAR(50), 98 departure_datetime DATETIME, 99 arrival_datetime DATETIME, 100 status VARCHAR(20) DEFAULT 'SCHEDULED', 101 FOREIGN KEY (plane_id) REFERENCES AIRPLANE(plane_id) 102); 103 104 CREATE TABLE SUPPLIER (105 supplier_id INT PRIMARY KEY AUTO_INCREMENT, 106 supplier_name VARCHAR(100) NOT NULL, 107 contact_person VARCHAR(100), 108 phone VARCHAR(20), 109 email VARCHAR(100), 110 address TEXT 111); 112 113 </pre>

5. Data Manipulation Language (DML) – Sample Data

The sample data (ercan_sample_data.sql) inserts meaningful records into all tables. Each table has at least 3 rows (most have 5+). The data simulates a realistic airport environment with:

- 5 airplane models
- 5 airplanes
- 5 hangars
- 3 hangar occupancy records (two currently inside)
- 5 test types
- 6 employees (4 technicians, 1 controller, 1 pilot)
- 4 technicians with expertise on different models
- 2 pilots with license and medical expiry
- 6 testing events
- 3 maintenance records
- 3 flight schedules
- 3 suppliers

<pre>1 • INSERT INTO MODEL (model_name, manufacturer, capacity) VALUES 2 ('Boeing 737', 'Boeing', 180); 3 ('Airbus A320', 'Airbus', 150); 4 ('Bombardier CRJ900', 'Bombardier', 90); 5 ('Airbus A380', 'Airbus', 500); 6 ('Boeing 787', 'Boeing', 290); 7 8 • INSERT INTO AIRPLANE (plane_no, model_id, status) VALUES 9 ('BVA-001', 1, 'ACTIVE'); 10 ('BVA-002', 1, 'MAINTENANCE'); 11 ('BVA-003', 2, 'ACTIVE'); 12 ('TC-001', 4, 'ACTIVE'); 13 ('BVA-004', 5, 'ACTIVE'); 14 15 • INSERT INTO HANGAR (hangar_no, location) VALUES 16 ('H001', 'North Terminal'); 17 ('H002', 'South Terminal'); 18 ('H003', 'Maintenance Bay'); 19 ('H004', 'East Terminal'); 20 ('H005', 'West Terminal'); 21 22 • INSERT INTO HANGAR_OCCUPANCY (plane_id, hangar_id, check_in_date, check_out_date) VALUES 23 (1, 1, '2025-05-01', '2025-05-08'); 24 (1, 2, '2025-05-01', NULL); 25 (2, 2, '2025-05-10', NULL); 26</pre>	<pre>27 • INSERT INTO TEST (test_name, description, max_score) VALUES 28 ('Engine Test', 'Check engine performance and thrust', 100); 29 ('Wing Inspection', 'Check wing integrity and surface', 100); 30 ('Avionics Check', 'Test electronic systems and navigation', 100); 31 ('Landing Gear', 'Check landing gear operation', 100); 32 ('Emergency Systems', 'Test emergency equipment', 100); 33 34 • INSERT INTO EMPLOYEE (emp_name, emp_type, union_no, hire_date) VALUES 35 ('Alpha Samura', 'TECHNICIAN', 'UNION001', '2020-01-15'); 36 ('Natair Barrow', 'TECHNICIAN', 'UNION002', '2021-03-20'); 37 ('Ebriina Khan', 'CONTROLLER', 'UNION003', '2019-11-10'); 38 ('Daniel Johnson', 'TECHNICIAN', 'UNION004', '2022-01-10'); 39 ('Fatima Hassan', 'TECHNICIAN', 'UNION005', '2021-11-20'); 40 ('Ousman Jallow', 'PILOT', 'UNION006', '2020-05-15'); 41 42 • INSERT INTO TECHNICIAN (emp_id, expertise_model_id, certification_date) VALUES 43 (1, 1, '2020-06-01'); 44 (2, 2, '2021-08-15'); 45 (4, 3, '2022-03-01'); 46 (5, 4, '2022-02-15'); 47</pre>
<pre>INSERT INTO PILOT (emp_id, license_number, flight_hours, medical_cert_expiry) VALUES (3, 'PIL-001', 2500, '2027-01-01'); (6, 'PIL-002', 1800, '2026-12-15'); INSERT INTO TESTING_EVENT (plane_id, test_id, tech_id, test_date, hours_spent, score) VALUES (1, 1, 1, '2025-05-15', 2.5, 95); (1, 2, 1, '2025-05-15', 1.5, 88); (3, 1, 2, '2025-05-14', 3.0, 92); (2, 3, 2, '2025-05-10', 2.0, 78); (4, 1, 3, '2025-05-12', 3.5, 90); (5, 2, 4, '2025-05-13', 2.5, 85); INSERT INTO MAINTENANCE_RECORD (plane_id, maintenance_type, start_date, end_date, cost, desc (2, 'Engine Overhaul', '2025-04-01', '2025-04-15', 50000.00, 'Full engine inspection and rep (1, 'Routine Check', '2025-05-01', '2025-05-02', 5000.00, 'Regular maintenance check'); (3, 'Avionics Update', '2025-05-10', '2025-05-12', 12000.00, 'Navigation system upgrade');</pre>	<pre>60 • INSERT INTO MAINTENANCE_RECORD (plane_id, maintenance_type, start_date, end_date, cost, desc 61 (2, 'Engine Overhaul', '2025-04-01', '2025-04-15', 50000.00, 'Full engine inspection and rep 62 (1, 'Routine Check', '2025-05-01', '2025-05-02', 5000.00, 'Regular maintenance check'); 63 (3, 'Avionics update', '2025-05-10', '2025-05-12', 12000.00, 'Navigation system upgrade'); 64 65 • INSERT INTO FLIGHT_SCHEDULE (flight_no, plane_id, departure_airport, arrival_airport, depart 66 ('TK123', 1, 'Istanbul', 'London', '2025-05-20 08:00:00', '2025-05-20 18:30:00', 'SCHEDULED') 67 ('KQ456', 4, 'Nairobi', 'Berlin', '2025-05-21 22:00:00', '2025-05-22 06:00:00', 'SCHEDULED') 68 ('KL789', 3, 'Amsterdam', 'Paris', '2025-05-22 09:30:00', '2025-05-22 11:00:00', 'SCHEDULED') 69 70 • INSERT INTO SUPPLIER (supplier_name, contact_person, phone, email, address) VALUES 71 ('LePasse Aviation Parts Ltd', 'Mohamed Ali', '+905456789', 'mohamed@lepass.com', 'LePasse 72 ('Cyprus Aero Supplies Co.', 'Ahmad Hassan', '+907654321', 'ahmad@cyprus.com', 'Sanayi Industri 73 ('Turkish Technic', 'Mehmet Demir', '+902123456', 'mehmet@turkishtechnic.com', 'Istanbul, Tu</pre>

6. Screenshots of All Tables

Run `SELECT * FROM <table_name>;` for each table and paste the results below.

MODEL

!	model_id	model_name	manufacturer	capacity
1		Boeing 737	Boeing	180
2		Airbus A320	Airbus	150
3		Bombardier CRJ900	Bombardier	90
4		Airbus A380	Airbus	500
5		Boeing 787	Boeing	290

AIRPLANE

!	plane_id	plane_no	model_id	status
1		5Y-KQA	1	ACTIVE
2		5Y-KQB	1	MAINTENANCE
3		ET-APK	2	ACTIVE

HANGAR

! hangar_id	hangar_no	location
1	H001	North Terminal
2	H002	South Terminal
3	H003	Maintenance Bay

HANGAR_OCCUPANCY

! occupancy_id	plane_id	hangar_id	check_in_date	check_out_date
1	1	1	2026-05-01	2026-05-10
2	2	3	2026-05-05	NULL

TEST

! test_id	test_name	description	max_score
1	Engine Test	Check engine performa...	100
2	Wing Inspection	Check wing integrity an...	100
3	Avionics Check	Test electronic systems ...	100

EMPLOYEE

! emp_id	emp_name	emp_type	union_no	hire_date
1	Alpha Samura	TECHNICIAN	UNION001	2020-01-15
2	Matarr Barrow	TECHNICIAN	UNION002	2021-03-20

TECHNICIAN

!	tech_id	emp_id	expertise_model_id	certification_date
1		1	1	2020-06-01
2		2	2	2021-08-15

PILOT

!	pilot_id	emp_id	license_number	flight_hours	medical_cert_expiry
1		3	PIL-001	2500	2027-01-01
2		6	PIL-002	1800	2026-12-15

TESTING_EVENT

!	event_id	plane_id	test_id	tech_id	test_date	hours_sp...	score
1		1	1	1	2026-05-15	2.5	95
2		1	2	1	2026-05-15	1.5	88

MAINTENANCE_RECORD

!	mainte...	plane_id	maintena...	start_date	end_date	cost	description
1		2	Engine Ov...	2026-04-01	2026-04-15	50000	Full engine inspe...
2		1	Routine Ch...	2026-05-01	2026-05-02	5000	Regular maintena...

FLIGHT_SCHEDULE

!	flight...	flight_no	plane_id	departu...	arrival_...	departu...	arrival_...	status
1		TK123	1	Istanbul	London	2026-05-...	2026-05-...	SCHEDULED
2		KQ456	4	Nairobi	Berlin	2026-05-...	2026-05-...	SCHEDULED
3		KL789	3	Amsterdam	Paris	2026-05-...	2026-05-...	SCHEDULED

SUPPLIER

!	supplier_id	supplier_name	contact_per...	phone	email	address
1		Lefkosa Aviati...	Mohammed Ali	+903456789	mohammed@...	Lefkosa Airport Road
2		Cyprus Aero S...	Ahmad Hassan	+907654321	ahmad@aero....	Sanayi Industrial Park
3		Turkish Technic	Mehmet Demir	+902123456	mehmet@turk...	Istanbul, Turkey

7. Queries for Management Statistics

The project requires 15 statistical queries. Below are 15 selected queries from the Queries.sql file. Each query is presented with its SQL code, and an explanation.

Query 1: Active Airplanes

```
SELECT plane_no, status FROM AIRPLANE WHERE status = 'ACTIVE';
```

Explanation: Lists all airplanes currently available for flight. Management uses this to assign planes to flight schedules.

Query 2: Planes Currently Inside Hangars

```
SELECT a.plane_no, h.hangar_no, ho.check_in_date
FROM HANGAR_OCCUPANCY ho
JOIN AIRPLANE a ON ho.plane_id = a.plane_id
JOIN HANGAR h ON ho.hangar_id = h.hangar_id
WHERE ho.check_out_date IS NULL;
```

Explanation: Shows which airplanes are still inside hangars (no check-out date). Helps hangar space management.

Query 3: Number of Tests Performed by Each Technician

```
SELECT t.tech_id, e.emp_name, COUNT(*) as tests_done
FROM TESTING_EVENT te
JOIN TECHNICIAN t ON te.tech_id = t.tech_id
JOIN EMPLOYEE e ON t.emp_id = e.emp_id
GROUP BY t.tech_id, e.emp_name;
```

Explanation: Counts tests per technician – used for performance evaluation and workload distribution.

Query 4: Average Test Score per Airplane

```
SELECT a.plane_no, AVG(te.score) as avg_score
FROM TESTING_EVENT te
JOIN AIRPLANE a ON te.plane_id = a.plane_id
GROUP BY a.plane_no;
```

Explanation: Calculates average test score for each airplane. Low averages may indicate maintenance needs.

Query 5: Technicians Expert in Boeing 737

```
SELECT e.emp_name  
FROM TECHNICIAN t  
JOIN EMPLOYEE e ON t.emp_id = e.emp_id  
JOIN MODEL m ON t.expertise_model_id = m.model_id  
WHERE m.model_name = 'Boeing 737';
```

Explanation: Finds technicians qualified for a specific model – useful when scheduling maintenance for Boeing 737 airplanes.

Query 6: Detailed Test Event List

```
SELECT a.plane_no, ts.test_name, e.emp_name, te.test_date, te.score  
FROM TESTING_EVENT te  
JOIN AIRPLANE a ON te.plane_id = a.plane_id  
JOIN TEST ts ON te.test_id = ts.test_id  
JOIN TECHNICIAN t ON te.tech_id = t.tech_id  
JOIN EMPLOYEE e ON t.emp_id = e.emp_id;
```

Explanation: Complete view of all test events, including plane, test type, technician, date, and score. Used for audit trails.

Query 7: Number of Airplanes per Model (including zero)

```
SELECT m.model_name, COUNT(a.plane_id) as plane_count  
FROM MODEL m  
LEFT JOIN AIRPLANE a ON m.model_id = a.model_id  
GROUP BY m.model_name;
```

Explanation: Shows how many airplanes belong to each model, even models with zero planes. Helps fleet composition analysis.

Query 8: Total Hours Spent on All Tests

```
SELECT SUM(hours_spent) as total_hours FROM TESTING_EVENT;
```

Explanation: Total technician hours spent on testing – important for labour cost calculation.

Query 9: Tests with Score >= 70 (Passing)

```
SELECT * FROM TESTING_EVENT WHERE score >= 70;
```

Explanation: Lists all passing test events. Management can quickly review successful tests.

Query 10: Employees Hired in the Last 2 Years

```
SELECT emp_name, hire_date  
FROM EMPLOYEE  
WHERE hire_date >= DATE_SUB(CURDATE(), INTERVAL 2 YEAR);
```

Explanation: Identifies recent hires – useful for HR and training planning.

Query 11: Airplanes That Have Never Been Tested

```
SELECT plane_no FROM AIRPLANE  
WHERE plane_id NOT IN (SELECT DISTINCT plane_id FROM TESTING_EVENT);
```

Explanation: Flags airplanes with no test history – a safety concern. Management must ensure all planes are tested.

Query 12: Top 3 Highest Scoring Tests

```
SELECT te.event_id, a.plane_no, te.score  
FROM TESTING_EVENT te  
JOIN AIRPLANE a ON te.plane_id = a.plane_id  
ORDER BY te.score DESC  
LIMIT 3;
```

Explanation: Highlights the best-performing test events – can be used to reward technicians or identify reliable planes.

Query 13: Number of Expert Technicians per Model

```
SELECT m.model_name, COUNT(t.tech_id) as expert_count  
FROM MODEL m  
LEFT JOIN TECHNICIAN t ON m.model_id = t.expertise_model_id
```

```
GROUP BY m.model_name;
```

Explanation: Shows how many technicians are certified for each model. Helps identify skill gaps.

Query 14: Pass/Fail Result per Test Event

```
SELECT te.event_id, a.plane_no, ts.test_name, te.score,  
       CASE WHEN te.score >= 70 THEN 'PASS' ELSE 'FAIL' END as result  
FROM TESTING_EVENT te  
JOIN AIRPLANE a ON te.plane_id = a.plane_id  
JOIN TEST ts ON te.test_id = ts.test_id;
```

Explanation: Adds a human-readable PASS/FAIL column. Management reports can use this directly.

Query 15: Monthly Test Statistics

```
SELECT YEAR(test_date) AS year, MONTH(test_date) AS month,  
       COUNT(*) AS tests_count, AVG(score) AS avg_score  
FROM TESTING_EVENT  
GROUP BY YEAR(test_date), MONTH(test_date)  
ORDER BY year DESC, month DESC;
```

Explanation: Trends of testing volume and average scores over time. Useful for identifying seasonal issues or improvements.

8. GitHub Repository

All project files are hosted on GitHub:

- **Repository URL:** https://github.com/22421230/Ercan_airport-management-database.git
- **Collaborators:** Matarr Barrow-22421230, Alpha Samura-22019986
- **Contents:**
 - ER diagram image (ERD.png)
 - ercan_schema.sql – DDL
 - ercan_sample_data.sql – DML sample data
 - Queries.sql – 20 queries, stored procedure, view
 - Ercan Airport Management System relational schema.txt
 - README.md – setup and execution instructions

The repository is public and accessible by the instructor.

9. Conclusion

The Ercan Airport Management database successfully implements all required features:

- 12 tables (exceeding minimum 8)
- Full DDL with primary keys, foreign keys, unique constraints, and check constraints
- Realistic sample data
- 20 statistical queries (15 presented in this report) covering joins, aggregations, subqueries, date functions, and case statements
- Additional advanced features: stored procedure and view

The database can be used by airport management to track airplane status, hangar occupancy, test results, technician expertise, and employee records. The system is extensible and can be integrated with a front-end application for daily operations.